

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Черников Кирилл

Группа
J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Цель работы

Мигрировать ранее разработанное приложение DataForge (ЛР1 и ЛР2) на фреймворк Vue.js 3 и реализовать полноценное одностраничное приложение (SPA) с клиентской маршрутизацией, взаимодействием с внешним API через axios, компонентным подходом и composable-функциями для выделения повторяющейся логики.

2 Индивидуальное задание

Разработать SPA-версию платформы для автоматизированной подготовки и аннотации данных для ML на основе Vue.js 3. Приложение должно включать шесть страниц (вход, регистрация, личный кабинет, поиск, задача, рабочее), клиентский роутер с защитой маршрутов и работу с моковым API на json-server.

Обязательные требования:

- Подключённый роутер (Vue Router) с navigation guards
- Взаимодействие с внешним API через axios
- Разумное деление на компоненты
- Использование composable-функций для повторяющегося функционала

3 Ход выполнения работы

3.1 Подготовка окружения

Проект создан на базе Vite с плагином @vitejs/plugin-vue. Структура зависимостей в package.json:

```
{
  "dependencies": {
    "axios": "1.7.2",
    "bootstrap": "5.3.3",
    "vue": "3.4.27",
    "vue-router": "4.3.2"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "5.0.4",
    "concurrently": "8.2.2",
    "json-server": "0.17.4",
    "vite": "5.2.11"
  }
}
```

Скрипт `npm run dev` одновременно запускает Vite на порту 5173 и json-server на порту 3000 через concurrently. Структура проекта:

```
lab3/
  index.html
  vite.config.js
  package.json
```

```

public/icons.svg
server/db.json
src/
  main.js
  App.vue
  assets/styles.css
  router/index.js
  composables/
    useApi.js
    useAuth.js
    useTheme.js
  components/
    AppNavbar.vue
    MetricCard.vue
    ProjectCard.vue
  views/
    LoginView.vue
    RegisterView.vue
    DashboardView.vue
    SearchView.vue
    TaskView.vue
    WorkersView.vue

```

3.2 Маршрутизация и navigation guards

Используется `createWebHashHistory` для совместимости с локальным сервером без серверной конфигурации. Определены семь маршрутов с метаданными `requiresAuth` и `guest`:

```

const routes = [
  { path: '/', redirect: '/login' },
  { path: '/login', component: LoginView,
    meta: { guest: true }, name: 'login' },
  { path: '/register', component: RegisterView,
    meta: { guest: true }, name: 'register' },
  { path: '/dashboard', component: DashboardView,
    meta: { requiresAuth: true }, name: 'dashboard' },
  { path: '/search', component: SearchView,
    meta: { requiresAuth: true }, name: 'search' },
  { path: '/task/:id', component: TaskView,
    meta: { requiresAuth: true }, name: 'task' },
  { path: '/workers', component: WorkersView,
    meta: { requiresAuth: true }, name: 'workers' },
]

```

Navigation guard проверяет сессию перед каждым переходом: неавторизованный пользователь перенаправляется на `/login`, авторизованный не может открыть `/login` или `/register`:

```

router.beforeEach((to) => {
  const { getSession } = useAuth()
  const session = getSession()

```

```

    if (to.meta.requiresAuth && !session) return { name: 'login' }
    if (to.meta.guest && session) return { name: 'dashboard' }
  })
})

```

3.3 Composable-функции

Для выделения повторяющейся логики реализованы три composable-функции.

useApi.js — обёртка над axios с обработкой ошибок сети:

```

import axios from 'axios'

const api = axios.create({ baseURL: 'http://localhost:3000' })

export function useApi() {
  async function get(path, params) {
    try {
      const response = await api.get(path, { params })
      return response.data
    } catch (error) {
      throw new Error(getErrorMessage(error))
    }
  }
  async function post(path, data) { ... }
  async function patch(path, data) { ... }
  return { get, post, patch }
}

```

useAuth.js — управление сессией в localStorage:

```

const SESSION_KEY = 'dataforge_session'

export function useAuth() {
  function getSession() {
    try {
      const raw = localStorage.getItem(SESSION_KEY)
      return raw ? JSON.parse(raw) : null
    } catch {
      localStorage.removeItem(SESSION_KEY)
      return null
    }
  }
  function setSession(user) {
    localStorage.setItem(SESSION_KEY, JSON.stringify({
      userId: user.id,
      token: 'mock-token-${user.id}-${Date.now()}',
    }))
  }
  function clearSession() {
    localStorage.removeItem(SESSION_KEY)
  }
  return { getSession, setSession, clearSession }
}

```

useTheme.js — переключение темы с сохранением в `localStorage`. Состояние `theme` объявлено как модульный реактивный `ref` вне функции, поэтому все компоненты разделяют одно значение:

```
const theme = ref(
  localStorage.getItem(THEME_KEY) ||
  (matchMedia('(prefers-color-scheme: dark)').matches
   ? 'dark' : 'light')
)

export function useTheme() {
  function applyTheme(value) {
    document.documentElement.setAttribute('data-theme', value)
    document.documentElement.setAttribute('data-bs-theme', value)
    theme.value = value
  }
  function toggleTheme() {
    const next = theme.value === 'dark' ? 'light' : 'dark'
    localStorage.setItem(THEME_KEY, next)
    applyTheme(next)
  }
  return { theme, initTheme, toggleTheme }
}
```

3.4 Компоненты

AppNavbar.vue — общая навигационная панель для всех страниц. Динамически показывает разные пункты меню в зависимости от наличия сессии, содержит кнопку переключения темы и кнопку выхода:

```
<template v-if="!session">
  <li><RouterLink to="/login">          </RouterLink></li>
  <li><RouterLink to="/register">        </RouterLink>
    </li>
</template>
<template v-else>
  <li><RouterLink to="/dashboard">        </RouterLink></li>
  <li><RouterLink to="/search">           </RouterLink></li>
  <li><RouterLink to="/task/1">           </RouterLink></li>
  <li><RouterLink to="/workers">         </RouterLink></li>
  <li><button @click="logout">            </button></li>
</template>
```

MetricCard.vue — переиспользуемая карточка для отображения числовой метрики. Принимает `label` и `value` через пропсы. Используется на страницах `Dashboard` (4 карточки) и `Workers` (3 карточки):

```
<template>
  <article class="metric-card p-4 h-100">
    <p class="text-secondary mb-1">{{ label }}</p>
    <h2 class="display-6 fw-bold mb-0" aria-live="polite">
      {{ value ?? '-' }}
    </h2>
  </article>
</template>
```

```

    </article>
  </template>

  <script setup>
  defineProps({
    label: { type: String, required: true },
    value: { type: [String, Number], default: null },
  })
  </script>

```

ProjectCard.vue — карточка проекта для страницы поиска. Принимает объект `project` и отображает его данные с форматированием даты:

```

<template>
  <article class="project-card p-4 bg-white">
    <div class="d-flex justify-content-between mb-2">
      <h3 class="h5 mb-0">{{ project.title }}</h3>
      <span :class="'badge text-bg-{{project.statusVariant}}'">
        {{ project.statusLabel }}
      </span>
    </div>
    <p class="text-secondary mb-2">
      : {{ project.type }} |
      : {{ formatDate(project.deadline) }} |
      : {{ project.worker }}
    </p>
    <RouterLink :to="'/task/{{project.taskId}}'"
      class="btn btn-sm btn-brand">

    </RouterLink>
  </article>
</template>

```

3.5 Страницы (Views)

Каждый `view` — отдельный `.vue`-файл. Данные загружаются в `onMounted` через `useApi`. В таблице ниже приведены страницы и ключевые API-запросы:

View	Маршрут	API-запросы
LoginView	/login	GET /users?email=...
RegisterView	/register	GET /users?email=..., POST /users
DashboardView	/dashboard	GET /users/:id, /projects, /tasks, /workers, /qualityStats/1
SearchView	/search	GET /projects
TaskView	/task/:id	GET /tasks/:id, GET /projects/:id, PATCH оба
WorkersView	/workers	GET /workers, GET /teamSummary/1, PATCH оба

Пример загрузки данных на странице Dashboard:

```

onMounted(async () => {
  const session = getSession()

```

```

const [user, projects, tasks, workers, stats] =
  await Promise.all([
    get(`/users/${session.userId}`),
    get('/projects'),
    get('/tasks'),
    get('/workers'),
    get('/qualityStats/1'),
  ])

metrics.value = {
  activeProjects: projects.length,
  activeTasks: tasks.reduce((s, t) => s + t.progressTotal, 0),
  averageQuality: formatPercent(average(
    projects.map(p => p.quality))),
  newWorkers: workers.filter(w => w.isNew).length,
}
})

```

Фильтрация на странице поиска реализована через `computed`, что позволяет Vue автоматически пересчитывать список при изменении любого фильтра без дополнительных вызовов API:

```

const filteredProjects = computed(() =>
  projects.value.filter(p => {
    const statusMatch =
      !filters.value.status || p.status === filters.value.status
    const typeMatch =
      !filters.value.type || p.typeCode === filters.value.type
    const workerMatch =
      !filters.value.worker ||
      p.worker.toLowerCase()
        .includes(filters.value.worker.toLowerCase())
    return statusMatch && typeMatch && workerMatch
  })
)

```

4 Скриншоты

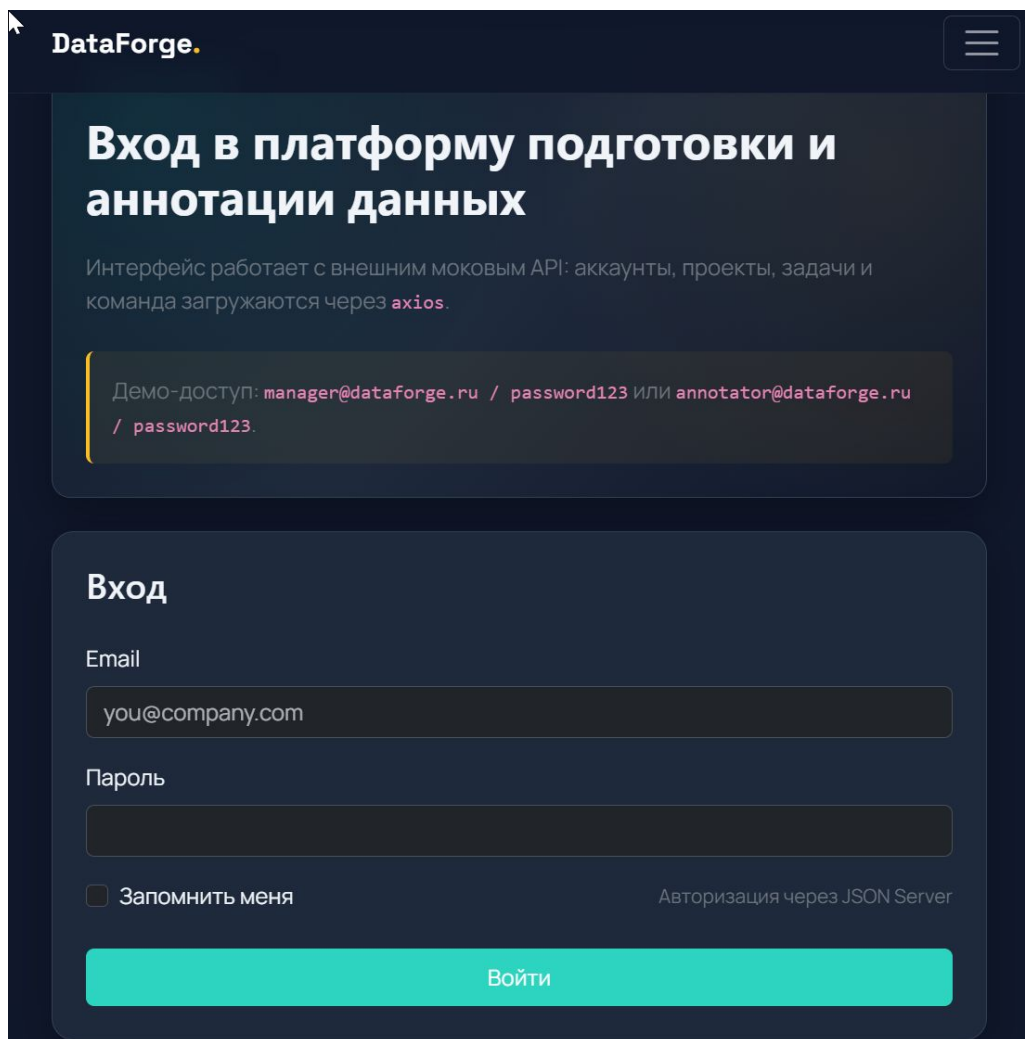


Рис. 1: Страница входа — форма авторизации в тёмной теме

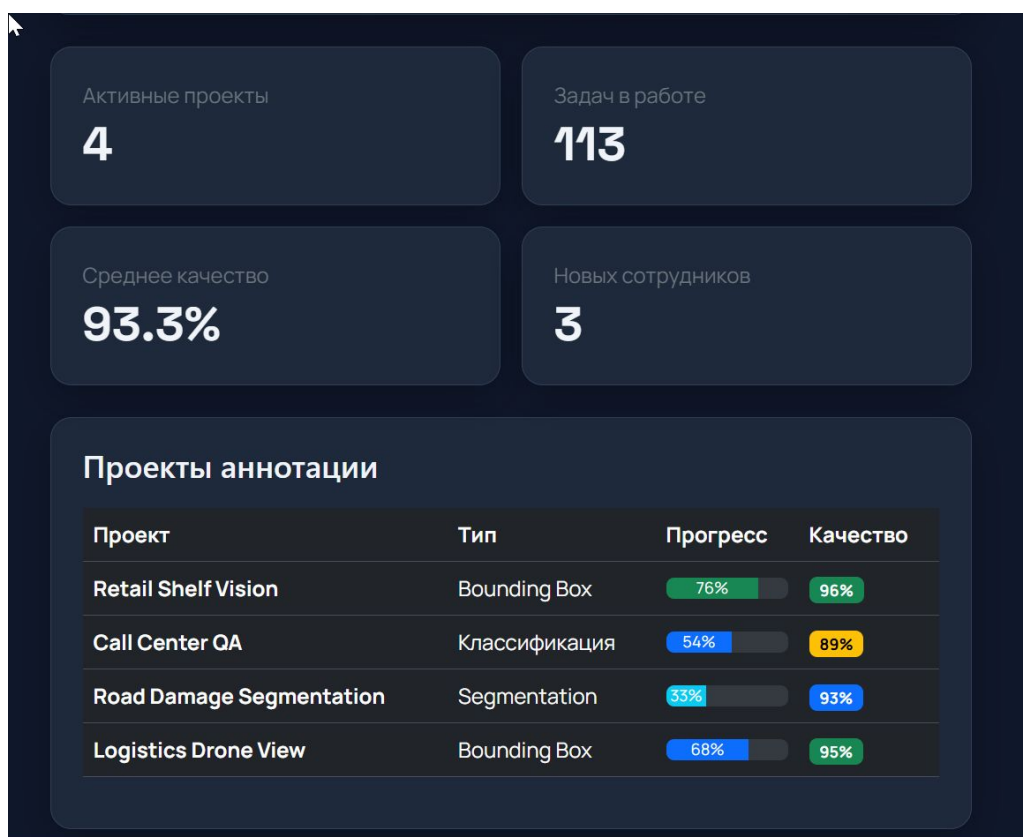


Рис. 2: Личный кабинет — 4 метрики (4 проекта, 113 задач, 93.3% качество, 3 новых рабочих) и таблица проектов из API

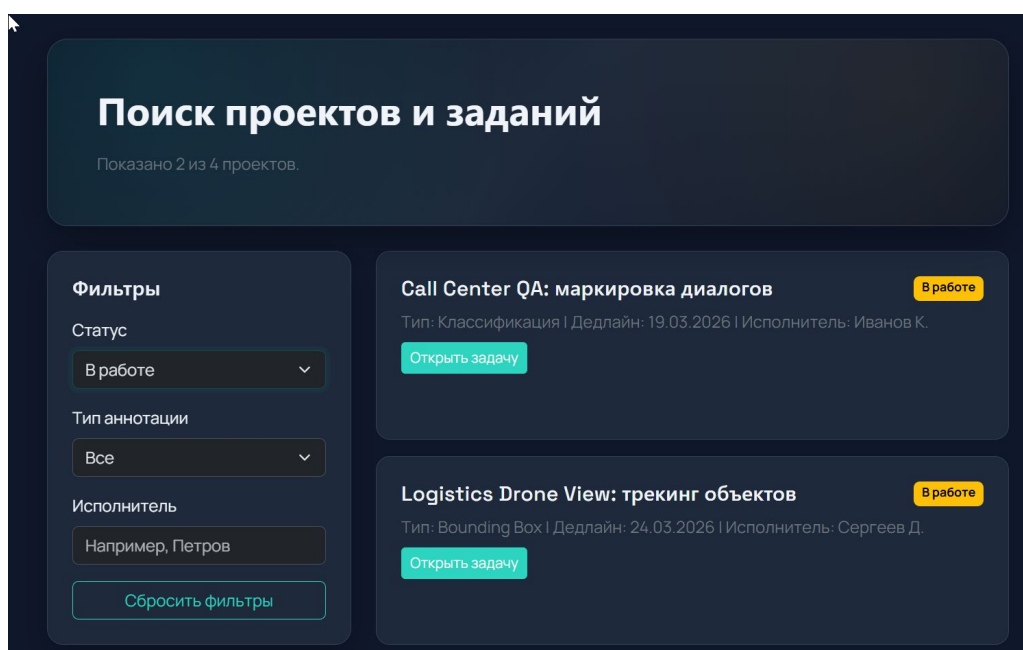


Рис. 3: Страница поиска — фильтр «В работе» показывает 2 из 4 проектов; реактивная фильтрация через computed без запросов к API

Страница задачи аннотации

Call Center QA / Диалог #CC-024

Отправить на проверку

Определите категорию обращения, тональность и необходимость эскалации.

Классификация

Теги

Комментарий

Активный инструмент: Классификация

Viewer area: данные для проекта "Call Center QA" загружены из API.

Инструкции

1. Укажите основную причину обращения клиента.
2. Отдельно отметьте эмоциональную окраску диалога.
3. При спорной классификации добавьте пояснение в комментарий.
4. Проверьте итоговые теги перед отправкой.

Статус

Исполнитель: Иванов К.

Рис. 4: Страница задачи — аннотация «Call Center QA / Диалог #CC-024» с инструментами классификации, тегов и комментариев

Управление рабочими

Распределение задач, валидация результатов и мониторинг метрик качества.

Автораспределение задач

Назначено задач

47

Средний QA score

93.5%

Ожидают валидации

18

Команда

Исполнитель	Роль	Текущая нагрузка	Качество	Действие
Петрова Анна	Исполнитель	12 задач	96%	Проверить
Иванов Кирилл	Валидатор	7 задач	88%	Назначить наставника
Смирнова Нина	Исполнитель	9 задач	92%	Проверить
Сергеев Дмитрий	Исполнитель	14 задач	95%	Понизить нагрузку

Рис. 5: Управление рабочими — 3 метрики (47 задач, 93.5% QA score, 18 ожидают валидации) и таблица команды с кнопкой автораспределения

5 Вывод

В ходе выполнения лабораторной работы приложение DataForge было полностью перенесено на фреймворк Vue.js 3. Реализован клиентский роутер на Vue Router с navigation guards, защищающими приватные маршруты. Взаимодействие с API перенесено с fetch на axios и изолировано в composable `useApi`. Логика авторизации и работы с темой также вынесена в отдельные composable-функции `useAuth` и `useTheme`. Интерфейс разбит на переиспользуемые компоненты: `AppNavbar`, `MetricCard` и `ProjectCard`. Реактивная фильтрация на странице поиска реализована через `computed` без дополнительных API-запросов. Все шесть страниц приложения функционируют корректно.